DNN Programming Assignment 2

Viradiya Tushar Rajeshbhai Student ID: 2026aj05073

Chosen settings: $h = 64$, $d = 64$; seeds $0, 1, 2$.

Problem 1: data, verification and HAR classification

UCI HAR uses nine channels in the prescribed order: body acceleration xyz, body gyroscope xyz, total acceleration xyz. Shapes are $7352 \times 128 \times 9$ (training source) and $2947 \times 128 \times 9$ (test), with labels remapped 1–6 to 0–5. The 21 training and 9 test subjects are disjoint. For each seed, NumPy's prescribed choice holds out four training subjects; normalization uses that seed's training portion only. Figure 1 shows class balance; constant channels are rejected.

The instructor permits freely chosen sizes and seeds; our choices are stated above. All recurrent classifiers use the last hidden state, one library layer, no dropout, Adam 10^{-3}, batch 128, 10 epochs and norm-1 clipping. Best checkpoints use validation macro-F1 (earliest tie). All reported spread is sample std over three seeds; timing is training-only CPU, four threads, not T4. Test data never select checkpoints.

Scratch cells use registered matrices, tanh RNN, LSTM i/f/g/o gates and GRU r/z/n gates, including PyTorch's reset on the recurrent candidate bias. Eight forward cases per cell cover two dtypes, two seeds and nonzero states. Maximum errors are $1.19 \cdot 10^{-7}$ (RNN), $2.38 \cdot 10^{-7}$ (LSTM/GRU), below 10^{-5}; backward errors are at most $2.22 \cdot 10^{-16}$. For $g = 1, 4, 3$ gates, single-bias count is $gh(d+h+1)$; PyTorch/custom count is $gh(d+h+2)$. At $d = 9, h = 64$, these are 4736/4800, 18944/19200 and 14208/14400. Classifier heads add $6h + 6$ parameters.

Table 1. HAR recurrent comparison

Model	Accuracy (%)	Macro-F1	Parameters	sec/epoch
RNN	70.52 ± 4.91	0.6814 ± 0.0495	5190	0.677
LSTM	87.71 ± 0.29	0.8768 ± 0.0023	19590	1.182
GRU	88.19 ± 1.04	0.8819 ± 0.0104	14790	2.279

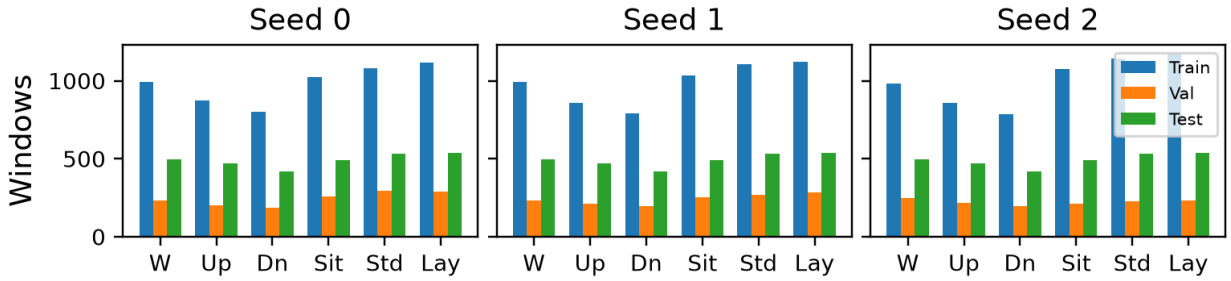


Figure 1. Class balance for all three subject splits and the test set.

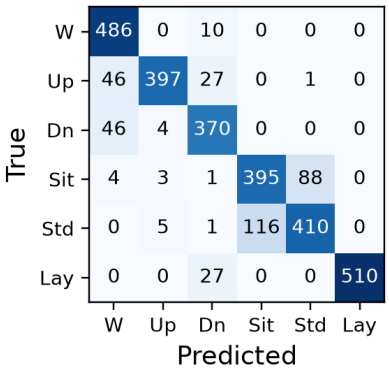


Figure 2. Validation-selected GRU, seed 2, epoch 9. Sitting/standing account for 204 cross-errors (88+116). Mean validation macro-F1 selects GRU (0.9104), then seed 2; it is not the highest-test-score run. Table 1 shows the strong classification gain from gating. The custom cells are checked against library layers; library recurrent layers are used only in permitted experiments.

Problem 1: adding problem, width and individual ablations

The supplied generator is unchanged: two markers, one per half; targets equal their marked-value sum. All $T \in \{50, 100, 200, 400\}$ use 10000/1000/1000 train/validation/test examples with data seeds 0/1/2. Models use $h = 64$, batch 128, Adam 10^{-3}, clipping 1 and five epochs; lowest validation MSE selects a checkpoint. Model seeds are 0/1/2. Solved means test MSE < 0.05. Table 2 and Figure 3a–b show no solved run (0/3 in every model/length group). The short budget was fixed before testing.

Table 2. Adding test MSE; every group solved 0/3

T	RNN	LSTM	GRU	Constant 1
50	0.1743 ± 0.0005	0.1743 ± 0.0003	0.1717 ± 0.0025	0.1754
100	0.1700 ± 0.0005	0.1700 ± 0.0008	0.1693 ± 0.0008	0.1697
200	0.1686 ± 0.0008	0.1684 ± 0.0005	0.1685 ± 0.0003	0.1679
400	0.1545 ± 0.0003	0.1538 ± 0.0005	0.1535 ± 0.0003	0.1527

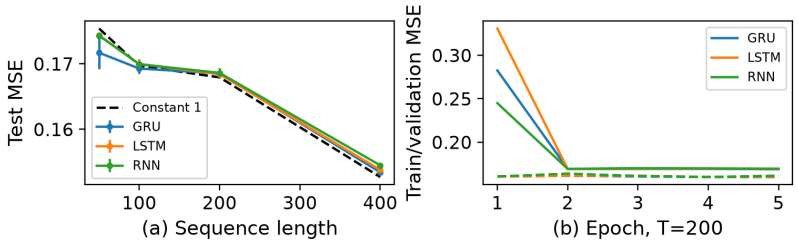


Figure 3a–b. (a) Test MSE versus length; (b) T=200 train (solid)/validation (dashed) curves. Table 3/Figure 4a retain the same 10-epoch HAR protocol; six $h = 64$ runs are reused. ID ends in 3 (odd): Table 4/Figure 4b compare initial LSTM effective forget bias 0/1 at T=200, five epochs, three paired seeds; the other weights and shuffle are identical.

Table 3. HAR sweep: LSTM (left), GRU (right)

h	LSTM acc. (%)	Params	sec	GRU acc. (%)	Params	sec
16	75.55 ± 6.53	1830	0.30	78.57 ± 2.23	1398	1.12
32	84.80 ± 1.12	5702	0.54	87.41 ± 0.73	4326	1.59
64	87.71 ± 0.29	19590	1.18	88.19 ± 1.04	14790	2.28
128	88.28 ± 0.43	71942	3.00	89.04 ± 0.70	54150	4.18

Table 4. Initial forget-bias ablation

Bias	Test MSE	Solved
0	0.16840 ± 0.00045	0/3
1	0.16849 ± 0.00039	0/3

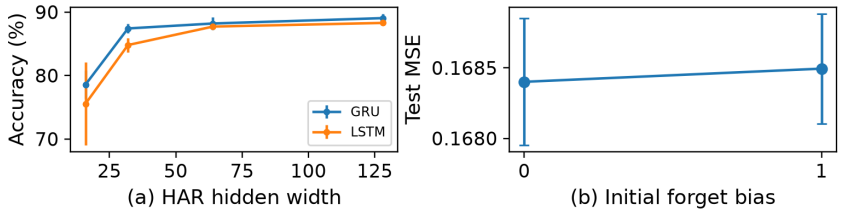


Figure 4a–b. (a) HAR width trade-offs; (b) initial forget-bias comparison.

Problem 1: gradient evidence and one-page analysis

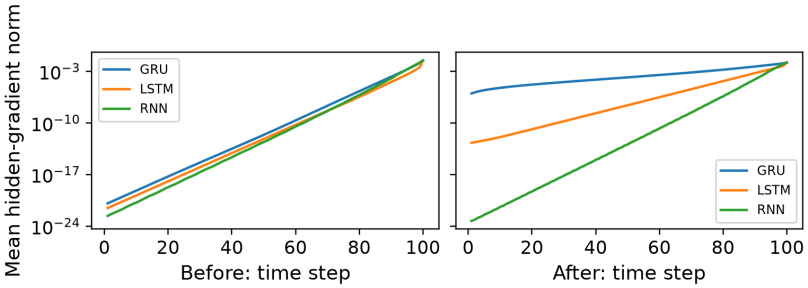


Figure 5. At T=100, mean batch L2 norms of actual $\partial L/\partial h_t$; log scale, before/after training, three-seed means. A fixed independent 32-example probe uses seed 3. Float64 custom unrolls reproduce library predictions and all recurrent parameter gradients. LSTM retains a separate cell-state graph.

Table 5. T=50 diagnostic, seed 0; test data never evaluated

Model	32-example train MSE	Val, 5 epochs	Val, 25 epochs
RNN	4.62e-06	0.16681	0.16463
LSTM	0.016	0.16587	0.01327
GRU	0.00187	0.16301	0.00189

Figure 5 shows early hidden-gradient attenuation at T=100. After training, the t=1/t=100 mean-norm ratios are about 3.6×10^{-22} (RNN), 1.2×10^{-11} (LSTM) and 6.5×10^{-5} (GRU). Predictions and parameter gradients match the library graph. Gating retains stronger early hidden signals here, but these derivatives also depend on loss and trajectory; dL/dh_t does not cover LSTM's separate cell-memory path or prove a general memory ranking.

Table 2 and Figure 3a–b preserve 36 unsolved five-epoch test runs. T=400's lower MSE also occurs for constant prediction, not better memory. New Table 5 reports independent T=50 diagnostics. All models fit 32 training examples below 0.05; memorization is not generalization. With unchanged data, initialization, shuffle, optimizer and batch, the 25-epoch pilots exactly replay their first five baseline epochs, then reach validation MSE 0.16463, 0.01327, 0.00189 (RNN/LSTM/GRU). This isolates duration as a gated short-sequence limitation. It neither explains T=200–400 nor establishes a new test ranking: no pilot test data were accessed. Finite gradients, target identity, state reset and parameter updates pass.

The paired odd-ID study changes only initial effective forget bias. Table 4/Figure 4b give test MSE 0.16840±0.00045 (bias 0) versus 0.16849±0.00039 (bias 1), neither solved. This difference is small relative to seed spread; no reliable benefit or harm follows under five epochs.

On HAR, Table 1 gives 70.52±4.91% (RNN), 87.71±0.29% (LSTM), 88.19±1.04% (GRU); Figure 2 shows sitting/standing confusion. Table 3/Figure 4a show GRU h 32→128 raises accuracy 87.41→89.04% while parameters grow 4326→54150. LSTM h 128 reaches 88.28% with 71942 parameters. CPU training favors LSTM over comparable GRU widths on this backend; it does not measure deployment energy.

For a memory-constrained wearable, these measurements favor prototyping GRU h 32:71% fewer parameters than h 64 for 0.78 points lower accuracy, and better accuracy than LSTM h 32. Float 32 weights occupy 16.9KiB before activations/runtime. Device latency and energy remain unmeasured; three seeds and short budgets limit certainty.

Problem 2: scratch Transformer and HAR encoder

Custom scaled dot-product attention computes

$$A = \mathrm{softmax}(QK^\top/\sqrt{d_k} + M), \qquad O = AV,$$

where forbidden logits are $-\infty$. Boolean True means visible. Fully masked rows are rejected. Row-sum, shape, causality and reference tests pass. Maximum SDPA reference error is $3.58 \cdot 10^{-7}$ and input-gradient error $8.88 \cdot 10^{-16}$. Only tests use framework attention/Transformer implementations.

Our nonuniform three-token example uses $Q = [(1,0);(0,1);(1,1)]$, $K = [(1,0);(0,1);(1,-1)]$, $V = [(1,0);(0,2);(3,4)]$. Let $a = 1/\sqrt{2}$, $E = e^a$. Scaled score rows are $(a,0,a),(0,a,-a),(a,a,0)$. Exponentiating and normalizing gives

$$A = \begin{bmatrix} E/(2E+1) & 1/(2E+1) & E/(2E+1) \\ 1/Z & E/Z & E^{-1}/Z \\ E/(2E+1) & E/(2E+1) & 1/(2E+1) \end{bmatrix}, \quad Z = 1 + E + E^{-1}.$$

Each output is $(A_{i1}+3A_{i3}, 2A_{i2}+4A_{i3})$; numerically $(1.6044, 2.0000), (0.7041, 1.7121), (0.9944, 1.5933)$. Scalar arithmetic and code agree below 10^{-5}. The earlier uniform/masked example and random reference tests remain in the notebook.

Biased Q/K/V/O projections split/merge heads. Blocks use pre-LayerNorm (eps= 10^{-5}), residuals, ReLU FFN width $4d$, dropout 0.1 on FFN activation and branch outputs. With dropout off, MHA/block reference errors are $1.80 \cdot 10^{-16}$ and $8.88 \cdot 10^{-16}$. Sinusoidal $PE_{p,2i} = \sin(p/10000^{2i/d})$, $PE_{p,2i+1} = \cos(p/10000^{2i/d})$ matches scalar tests within $5.96 \cdot 10^{-8}$; backward/causality tests pass.

At our $d = 64$, $d_{ff} = 256$, block matrices contribute $4(64)^2 + 2(64)(256) = 16384 + 32768 = 49152$ weights. Attention biases add $4(64) = 256$; FFN biases $256 + 64 = 320$; two LayerNorms add $4(64) = 256$. Total $49152 + 256 + 320 + 256 = 49984$, matching actual parameters. The general formula is $12d^2 + 13d$. Two blocks, patch embedding, CLS, final norm and 6-class head give 105222; learned positions add 1088. The untied65-character LM has 108481 parameters.

Encoders in Figure 6 use8-step patches (72 values), a $d = 64$ embedding, CLS, two blocks/four heads, then normalized CLS classification. Table 6 compares nine runs with none/sinusoidal/learned positions, using the exact P1 subjects, frozen training statistics, batch 128, Adam10^{-3}, clipping 1 and 10 epochs. Architecture, dropout and patching differ from recurrent models.

Table 6. HAR positional comparison, three seeds

Positions/model	Accuracy (%)	Macro-F1	Params	sec/epoch
learned	91.20 ± 0.53	0.9121 ± 0.0057	106310	1.521
none	90.54 ± 1.82	0.9045 ± 0.0182	105222	1.419
sinusoidal	90.61 ± 1.02	0.9062 ± 0.0106	105222	1.544
P1 GRU	88.19 ± 1.04	0.8819 ± 0.0104	14790	2.279

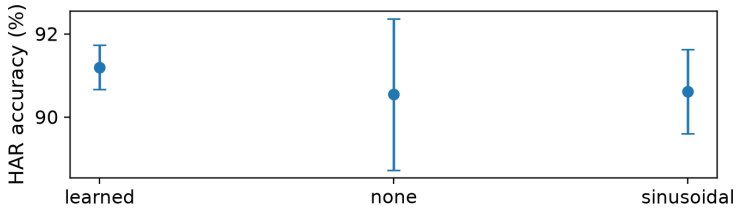


Figure 6. HAR accuracy with sample std. Mean validation F1 chooses sinusoidal (0.9276), not learned (0.9144), despite learned's higher test mean. The selected encoder improves accuracy by 2.42 points over GRU, using 7.11× the parameters. Flattening preserves the eight ordered offsets inside each patch, even without positions. The trained no-position probe is invariant to whole-patch reversal (max logit change $1.91 \cdot 10^{-6}$), but within-patch reversal changes logits by 5.52. Depth, size, patching and dropout confound attention-specific attribution. CPU time is not energy.

Problem 2: causal language modeling, heads and masking

Tiny Shakespeare has 1115394 characters and 65 distinct characters. Sorted vocabulary uses the whole text; contiguous 90/10 split gives 1003854/111540 training/validation characters. Contiguous nonoverlapping 128-step next-character blocks are not shuffled; fewer than 128 trailing transitions per split are omitted. No targets cross the split boundary. CPU uses the assignment FAQ exception: $d = 64$, two layers (rather than $d = 128$/four), context 128/four heads. All comparisons use five full epochs, batch 64, seed 0, Adam10^{-3}, dropout 0.1 and norm1 clipping. Minimum validation loss selects checkpoints; Figure 7 shows continued learning. Head/mask variants start with identical parameter values; all have 108481 parameters.

Table 7. LM validation; BPC = nats / ln 2

Heads	Mask	5-epoch nats	Continued nats	Continued BPC
4	causal	2.23495	2.02695	2.92427
1	causal	2.20279	1.99610	2.87976
8	causal	2.25598	2.07216	2.98949
4	leakage	0.31769	0.03194	0.04608

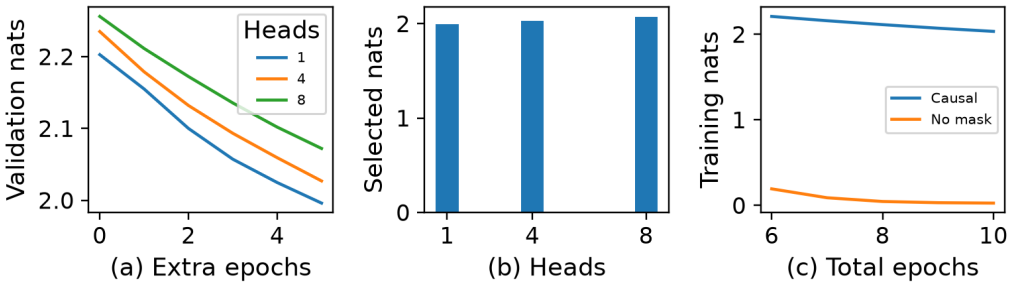


Figure 7a–c. (a) Continued validation curves; (b) selected causal losses; (c) mask-removal training loss. Original five-epoch curves remain in the notebook. Uniform baseline is $\ln 65 = 4.17439$ nats. Five additional epochs improve causal validation (Table 7). All variants restart Adam because old optimizer states are absent; duration and restart effects are not isolated. Original ablations have matched initialization; continuations start from their own validation-selected weights under a common rule. One seed cannot establish a ranking. Heads share the same matrix counts but store different numbers of softmax maps.

In the original five-epoch mask ablation, input $t + 1$ exposes the target at t. Training loss falls to 0.59362 versus causal 2.26579. Apparent validation 0.31769 nats/0.45834 BPC is **contaminated**, not autoregressive quality. Changing future tokens leaves the trained causal prefix logits unchanged (max 0), but changes unmasked logits by 7.15354. Its original generation exposes failure: 98.4% of its 500 characters are newlines and only 4 characters are distinct (temperature 0.8).

The primary LM is the continued four-head causal checkpoint (Table 7), selected at additional epoch 5. Its three 500-character draws are in `problem2/generated_samples.txt`: prompt `ROMEO:\n`, top-k 20, seed 700, temperatures 0.2/0.8/1.5. Table 8 describes single draws, not predictive entropy.

Table 8. Continued main-model sample diversity

Temperature	Distinct chars	Empirical entropy (bits)
0.2	23	3.402
0.8	38	4.355
1.5	50	4.782

At 0.2, “the he”/“and the” recur; “halll” shows spelling failure despite common words. At 0.8, “That pray here you sliang” has clause-like shape but malformed words and no coherent continuation. At 1.5, “LRIENCUS:” resembles a speaker cue, surrounded by broken words such as “caingmiely”. Increased case/punctuation variety is not improved coherence. All three samples remain weak despite lower validation loss.

Problem 2: attention interpretation and context scaling

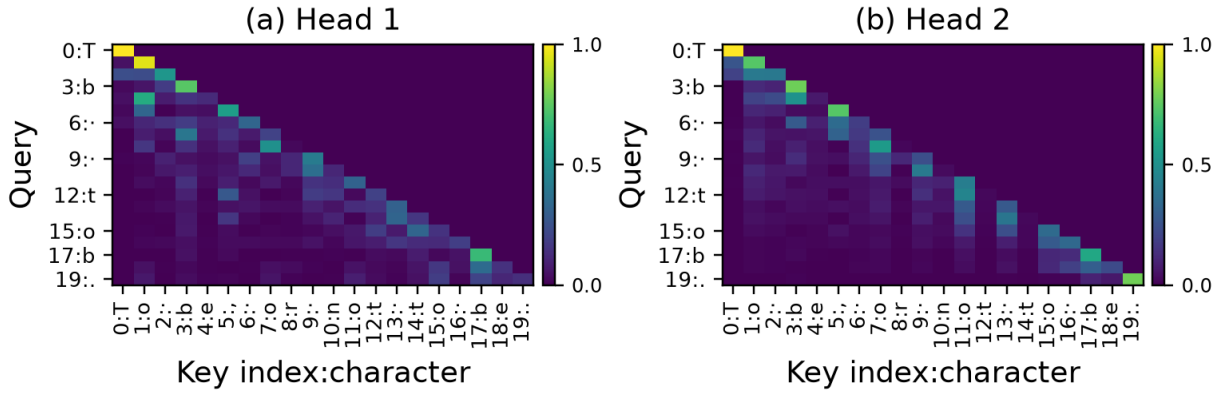


Figure 8a–b. Five-epoch baseline layer 1, heads 1/2 on "To be, or not to be." In Figure 8, rows are queries, columns keys. Both favor current/nearby characters with different off-diagonal patterns (mean diagonal mass 0.3616/0.3736). At query 17 (b), head 1 assigns 0.6783 to itself and 0.0460 to key 3 (earlier b); head 1's query 18 (e) places 0.3480 on key 17 (b), linking the local be pair. For query 18, head 2 places 0.2982 on key 17 and 0.1891 on the preceding space; query 17 attends 0.5991 to itself and 0.1213 to that space. Such weights describe this input and do not establish semantic importance or a causal mechanism. Future entries are exactly zero. This single sentence does not establish specialization.

Figure 9 and Table 9 measure baseline cost: batch 8, float 32, dropout off, contexts 64/128/256/512. Two warmups precede five timed forward+backward repeats; all raw times and three independent allocation-event traces per length are retained internally; the notebook replays matching measurements or measures a fresh run. CPU memory reconstructs peak additional live tensor bytes from raw profiler allocation/free events, relative to existing model/input baseline; it excludes interpreter/model weights and allocator reservation. It is neither process RSS nor reserved GPU memory.

Table 9. Full-model context cost on CPU

Context	Time (ms), mean ± std	Peak tensor MiB	Score elements
64	8.709 ± 0.402	5.228	131072
128	14.602 ± 0.810	12.714	524288
256	45.767 ± 0.897	41.331	2097152
512	187.750 ± 6.799	146.753	8388608

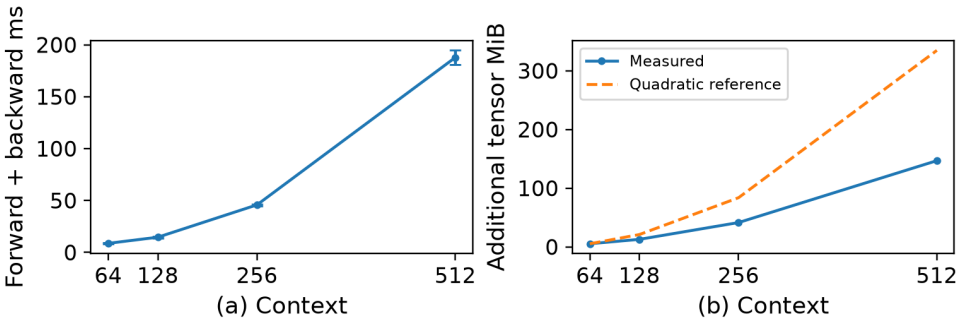


Figure 9a–b. Time and incremental tensor peak versus context; dashed line is an anchored quadratic memory reference. With 8× context, score elements increase 64×, measured time 21.56×, tensor peak 28.07×. Attention products cost $O(T^2d)$; embeddings/FFN add linear-T work, so full-model ratios need not equal 64. Short-length timing includes overhead and cache effects. Long-context quality and T4/device latency are not evaluated.

The 92 rows retain 82 baseline runs, 6 adding diagnostics and 4 LM continuations. Both notebooks reproduce these results; cached replay is not fresh training. No bonus claimed.

AI-use statement and references (excluded from six-page limit)

OpenAI Codex (including Codex CLI) assisted with requirements review, data pipelines, RNN, LSTM and GRU cells, Transformer scratch components, training and experiment code, tests, debugging, execution, plots, analysis/report drafting, reproducibility audits and packaging. This includes AI-written from-scratch code; no independent student authorship is claimed for that code. The student reports informal professor permission for AI assistance because of time constraints. Its scope is not independently verified and is not treated as a written exemption from the original scratch-code restrictions. The separate instructor email changes sizes, seeds and the attention example only. All experimental results are measured; unsuccessful runs and limitations are retained.

[1] A. Vaswani et al. *Attention Is All You Need.*2017. Sections 3.2.1,3.2.2,3.5 and Table 1: attention, head projections, positions and complexity. https://arxiv.org/abs/1706.03762. The assignment's pre-LN requirement is followed; the original paper uses post-LN.

[2] UCI Machine Learning Repository. *Human Activity Recognition Using Smartphones.* Official nine-channel sensor windows and subject IDs. https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones.

[3] A. Karpathy. *char-rnn: Tiny Shakespeare input.txt.* https://github.com/karpathy/char-rnn/tree/master/data/tinyshakespeare.

[4] PyTorch reference documentation and installed class documentation: RNN, LSTM, GRU, functional scaled_dot_product_attention, MultiheadAttention and TransformerEncoderLayer (verification only). https://pytorch.org/docs/stable/nn.html.

[5] Course assignment: *DNN Programming Assignment–2: RNN, LSTM, GRU and Transformer*, supplied DOCX. Source of dataset/split/generator instructions, marking criteria and CPU exception. The instructor clarification email supplied by the user overrides the size/seed table and requests a self-created three-token example. It confirms the due date as Saturday14 November 2026; the portal link is not supplied.

Reproducibility: Python 3.14.4, PyTorch 2.15.0+cpu and exact package pins in requirements.txt; four CPU threads. Original P2 training-only time 10.22min; P1 adding 25.27min; these exclude validation/setup and do not estimate T4 runtime. Checkpoints/raw data are intentionally excluded from the submission and regenerated by the notebooks.